AZURE DATA FACTORY · GUIDE N° 3 · PAS À PAS · NIVEAU EXPERT

Rejouer le temps

Un pipeline qui se rattrape seul, interroge l'État et facture les jours fériés

Un déclencheur à fenêtre bascule découpe le temps en journées de 24 heures et les rejoue une par une, dans l'ordre, même dans le passé. Chaque fenêtre demande à l'API publique du calendrier si le jour est férié, bascule sur une copie locale si l'API ne répond pas, écarte les sessions déjà facturées la veille, puis facture chaque recharge et chiffre la remise accordée.

Fenêtre bascule · Rattrapage automatique · Dépendance sur lui-même · Paramètres de pipeline · Web · Nouvelles tentatives · Try / catch / proceed · Existe · Jointure · Paramètre booléen · Variable booléenne

Par **Abdoul Hamid Diallo** · Microsoft Certified Fabric Data Engineer, Fabric Analytics Engineer et Azure AI Engineer

3 fenêtres
lancées seules, dans l'ordre

99 sessions
facturées sur 106 reçues

7 doublons
reconnus et écartés

89,05 €
de remise le 14 juillet

Rappel : on reste dans le même environnement. Ce guide réutilise **exactement** les ressources des guides n° 1 et n° 2 : le groupe de ressources `rg-adf-charge`, la fabrique `adf-charge-XXXX`, le compte de stockage `stchargeXXXX`, le service lié `LS_ADLS_Charge` (identité managée) et les datasets `DS_ADLS_Csv` et `DS_ADLS_Json`. Aucun dataset à créer : on ajoute un conteneur, un data flow, un pipeline et un déclencheur. Vous partez de zéro ? Faites d'abord les étapes 1 et 2 du guide n° 1, puis l'étape 2 du guide n° 2.

📎 Les 6 pièces jointes de ce PDF

Ouvrez le panneau **Pièces jointes** (le trombone) de votre lecteur PDF et enregistrez les **six fichiers** : chacun a sa place précise dans le conteneur `guide3`.

Pièce jointe	Dossier de dépôt	Ce qu'elle contient
`tariffs.json`	`reference`	la grille des 3 stations : heures pleines, heures creuses, frais de session
`holidays_2026.json`	`reference`	la copie de secours des jours fériés 2026, identique à l'API
`sessions_2026-07-13.json`	`raw`	lundi 13 juillet : 36 sessions
`sessions_2026-07-14.json`	`raw`	mardi 14 juillet, jour férié : 34 sessions, dont 4 renvoyées de la veille
`sessions_2026-07-15.json`	`raw`	mercredi 15 juillet : 36 sessions, dont 3 renvoyées de la veille
`sessions.csv`	`silver/2026-07-12`	le « jour zéro » : une seule ligne d'en-tête, la veille du premier jour

Le scénario

Les bornes de Metz, Nancy et Thionville envoient chaque jour un export JSON de leurs sessions. Deux pièges les attendent : les sessions commencées tard le soir sont **renvoyées** le lendemain, et le **14 juillet** est férié, donc facturé en heures creuses toute la journée.

Il faut facturer chaque jour sans compter deux fois une session, appliquer le bon tarif au bon jour, et pouvoir rejouer n'importe quelle journée passée sans rien casser.

Comment lire ce guide

Chaque valeur à saisir est dans un **bloc sombre**, à copier telle quelle. Une valeur qui commence par `@` se colle via **Ajouter du contenu dynamique**. En français : Web est dans **Général** ; Exists = **Existe**, Join = **Jointure**, Aggregate = **Agréger** ; Tumbling window = **Fenêtre bascule**.

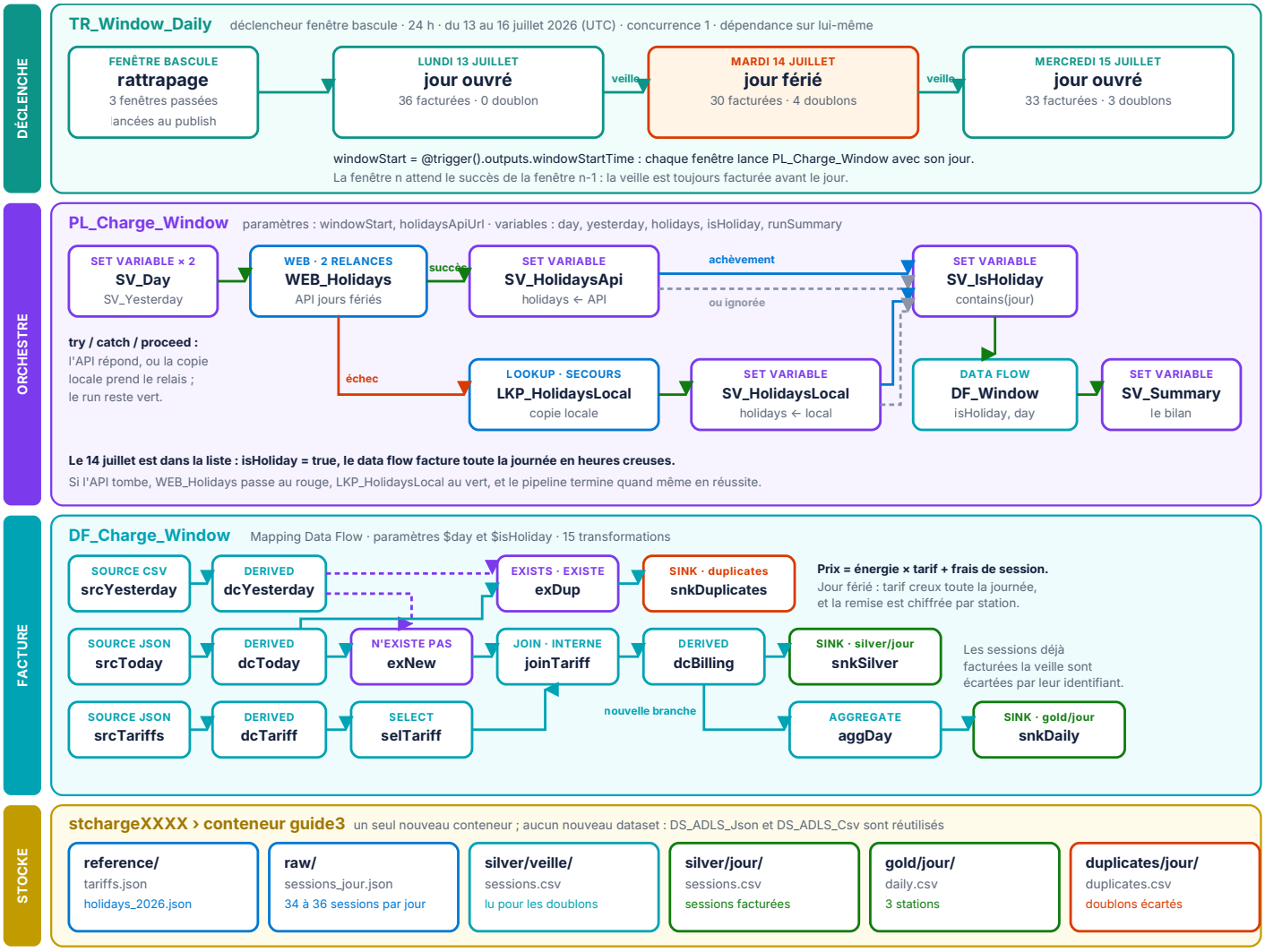
Le parcours

1. Préparer le conteneur guide3
2. Le data flow : le jour, la veille et la grille
3. Le data flow : dédoublonner, joindre, facturer
4. Le pipeline : la fenêtre et l'API avec son secours
5. Le déclencheur à fenêtre bascule
6. Vérifier, puis couper l'API pour éprouver le secours
7. Tout fermer

Ce que ce guide ajoute aux deux premiers. Le guide n° 1 transformait, le n° 2 jugeait. Celui-ci **maîtrise le temps** : il traite des intervalles, rattrape le passé, garantit l'ordre des jours, dialogue avec une API externe et survit à sa panne.

La carte complète du pipeline

Quatre couches, de haut en bas : **ce qui déclenche**, **ce qui orchestre**, **ce qui facture**, **où vivent les données**. Les nombres sont ceux que vous retrouverez à l'étape 6.



sarcelle : déclenchement dans le temps · violet : contrôle et décisions · bleu : lecture et API · turquoise : transformation Spark · vert : écriture, succès · orange : échec, doublons · or : stockage

Le temps en tranches
La fenêtre bascule transmet le début de chaque journée au pipeline. Publiée après coup, elle rattrape seule les jours passés.

L'API, et son secours
Web interroge le calendrier officiel, avec deux nouvelles tentatives. S'il échoue, la copie locale prend le relais et le run reste vert.

Existe, dans les deux sens
N'existe pas garde les sessions nouvelles ; Existe isole les doublons. Rien ne se perd : 34 = 30 + 4.

La veille avant le jour
La dépendance sur lui-même interdit au 14 de démarrer tant que le 13 n'a pas réussi : les doublons du 14 se calculent contre le 13.

1 Préparer le conteneur guide3

Portail Azure › **stchargeXXXX** › **Stockage des données** › **Conteneurs** › **+ Conteneur** :

```
guide3
```

Ouvrez **guide3** › **Charger**, dépliez **Avancé**, remplissez **Charger dans le dossier** (les sous-dossiers se créent seuls) et déposez les **six pièces jointes** selon le tableau de la page 1 : deux dans `reference`, trois dans `raw`, une dans `silver/2026-07-12`.

Pourquoi un jour zéro ? Chaque fenêtre relit les sessions facturées la veille. La première, le 13, cherche donc le 12 : ce fichier qui ne contient que l'en-tête lui répond « rien la veille », au lieu d'une erreur de chemin introuvable.

2 Le data flow : le jour, la veille et la grille

ADF Studio › **Auteur** › **+** › **Flux de données**. Laissez **Débogage du flux de données** éteint. Toile vide › onglet **Paramètres** › **+ Nouveau** deux fois.

nom du data flow	day · string · défaut	isHoliday · boolean · défaut
`DF_Charge_Window`	`'2026-07-13'`	`false()`

srcToday › dcToday

Source JSON › Colonne dérivée

Ajouter une source · **DS_ADLS_Json** · **Autoriser la dérive de schéma** coché · **Options de la source** › **Paramètres JSON** › **Forme du document** : **Tableau de documents**. Puis **+** › **Colonne dérivée dcToday**, quatre colonnes :

sid
```
toString(byName('session_id'))
```

start_hour
```
toInteger(substring(toString(byName('start_time')), 12, 2))
```

station_name
```
toString(byName('station'))
```

energy
```
toDouble(byName('energy_kwh'))
```

srcYesterday › dcYesterday

Source CSV › Dérivée

Sous srcToday : **DS_ADLS_Csv**, dérive cochée. Puis une colonne :

y_sid
```
toString(byName('session_id'))
```

srcTariffs › dcTariff › selTariff

comme au guide n° 2

DS_ADLS_Json, dérive cochée, **Tableau de documents**. Six colonnes typées, puis **Sélectionner** avec **Mappage automatique** décoché.

dcTariff	expression	dcTariff	expression
`tariff_station`	`toString(byName('station'))`	`peak_start`	`toInteger(byName('peak_start'))`
`peak_eur`	`toDouble(byName('peak_eur_kwh'))`	`peak_end`	`toInteger(byName('peak_end'))`
`offpeak_eur`	`toDouble(byName('offpeak_eur_kwh'))`	`session_fee`	`toDouble(byName('session_fee'))`

3 Le data flow : dédoublonner, joindre, facturer

exNew

Existe · N'existe pas

Le **+** de dcToday › **Existe**. Flux de droite **dcYesterday** · type **N'existe pas** · condition **sid** `==` **y_sid**. Les sessions jamais vues la veille.

exDup › snkDuplicates

Nouvelle branche · Existe

Le **+** de dcToday › **Nouvelle branche** › **Existe**. Même flux de droite, même condition, type **Existe**. Les doublons, tracés.

Côte à côte, jamais l'un derrière l'autre. exNew et exDup ont tous deux **dcToday** comme **Flux de gauche**. Chaînés, le premier jour ne laisserait passer aucune ligne.

joinTariff › dcBilling

Jointure interne › Colonne dérivée

Le **+** d'exNew › **Jointure** : flux de droite **selTariff**, type **Interne**, condition **station_name** `==` **tariff_station**. Puis **dcBilling**, cinq colonnes : `$isHoliday` fait passer toute la journée au tarif creux.

tariff_period
```
iif($isHoliday, 'FERIE', iif(start_hour >= peak_start && start_hour < peak_end, 'PLEINE', 'CREUSE'))
```

billed_eur
```
round(energy * iif($isHoliday, offpeak_eur, iif(start_hour >= peak_start && start_hour < peak_end, peak_eur, offpeak_eur)) + session_fee, 2)
```

workday_eur
```
round(energy * iif(start_hour >= peak_start && start_hour < peak_end, peak_eur, offpeak_eur) + session_fee, 2)
```

service_day
```
$day
```

day_type
```
iif($isHoliday, 'FERIE', 'OUVRE')
```

aggDay

Nouvelle branche · Agréger

Le **+** de dcBilling › **Nouvelle branche** › **Agréger**. Regrouper par **service_day**, **day_type**, **station_name** ; cinq agrégats :

sessions
```
count()
```

kwh_pleine
```
round(sum(toDouble(iif(tariff_period == 'PLEINE', toString(energy), '0'))), 2)
```

kwh
```
round(sum(energy), 2)
```

holiday_discount_eur
```
round(sum(workday_eur) - sum(billed_eur), 2)
```

revenue_eur
```
round(sum(billed_eur), 2)
```

Récepteur	Derrière	Réglages communs : DS_ADLS_Csv · Sortie dans un seul fichier · Effacer le dossier · Optimiser : Partition unique
snkSilver	dcBilling	fichier `sessions.csv`
snkDaily	aggDay	fichier `daily.csv`
snkDuplicates	exDup	fichier `duplicates.csv`

Exists, l'outil exact du dédoublonnage. Contrairement à une jointure, il ne rapporte aucune colonne : il teste seulement la présence. Sa paire Existe / N'existe pas sépare un flux en deux sans rien perdre ni rien dupliquer.

4 Le pipeline : la fenêtre et l'API avec son secours

Auteur › **+** › **Pipeline**. Onglet **Paramètres** : deux paramètres **String**. Onglet **Variables** : quatre **String** et une **Boolean**.

pipeline: `PL_Charge_Window`
paramètre 1: `windowStart`
défaut: `2026-07-13T00:00:00Z`
paramètre 2 : holidaysApiUrl, défaut: `https://calendrier.api.gouv.fr/jours-feries/metropole/2026.json`

String: `day` | String: `yesterday` | String: `holidays` | String: `runSummary` | Boolean: `isHoliday`

SV_Day

Définir une variable · day

```
@formatDateTime(pipeline().parameters.windowStart, 'yyyy-MM-dd')
```

SV_Yesterday

Définir une variable · yesterday

```
@formatDateTime(addDays(pipeline().parameters.windowStart, -1), 'yyyy-MM-dd')
```

WEB_Holidays

Web · GET

Relié en vert à SV_Yesterday. Onglet **Paramètres** : **Méthode GET**, **URL** en contenu dynamique. Onglet **Général** : le délai d'une activité Web vaut 10 minutes au minimum, l'intervalle entre deux tentatives 30 secondes au minimum.

URL: `@pipeline().parameters.holidaysApiUrl`

Délai d'expiration: `0.00:10:00` | Réessayer: `2` | Intervalle (s): `30`

SV_HolidaysApi

flèche verte · holidays

```
@string(activity('WEB_Holidays').output)
```

LKP_HolidaysLocal › SV_HolidaysLocal

flèche rouge · secours

Recherche · **DS_ADLS_Json** · `guide3` · `reference` · `holidays_2026.json` · **Première ligne uniquement**. Puis, variable **holidays** :

```
@string(activity('LKP_HolidaysLocal').output.firstRow)
```

SV_IsHoliday

Définir une variable · isHoliday · quatre flèches

Depuis **SV_HolidaysApi** puis depuis **SV_HolidaysLocal**, tirez la flèche **bleue** (À l'achèvement) et la flèche **grise** (Lors de l'omission) vers SV_IsHoliday.

```
@contains(json(variables('holidays')), variables('day'))
```

Try / catch / proceed, la logique des flèches. Deux flèches qui partent de la **même** activité valent « l'une OU l'autre » ; deux activités **différentes** valent « l'une ET l'autre ». Si l'API répond, la branche de secours est omise ; si elle échoue, c'est la branche API qui est omise. Dans les deux cas SV_IsHoliday s'exécute, et le pipeline finit en **Réussite** : un échec rattrapé n'est pas un échec.

DF_Window

Flux de données · DF_Charge_Window · Petit

Flux	container	folder	file
srcToday	`guide3`	`raw`	`@concat('sessions_', variables('day'), '.json')`
srcYesterday	`guide3`	`@concat('silver/', variables('yesterday'))`	`@string('')`
srcTariffs	`guide3`	`reference`	`tariffs.json`
snkSilver	`guide3`	`@concat('silver/', variables('day'))`	`@string('')`
snkDaily	`guide3`	`@concat('gold/', variables('day'))`	`@string('')`
snkDuplicates	`guide3`	`@concat('duplicates/', variables('day'))`	`@string('')`

day (expression de pipeline)

```
@variables('day')
```

isHoliday (expression de pipeline)

```
@variables('isHoliday')
```

SV_Summary

Définir une variable · runSummary

```
@concat('Fenêtre ', variables('day'), if(variables('isHoliday'), ' (jour férié)', ' (jour ouvré)'), ' : ',
string(activity('DF_Window').output.runStatus.metrics.snkSilver.rowsWritten), ' sessions facturées, ',
string(if(contains(activity('DF_Window').output.runStatus.metrics, 'snkDuplicates'),
activity('DF_Window').output.runStatus.metrics.snkDuplicates.rowsWritten, 0)), ' doublons écartés')
```

Tout valider, puis **Déboguer** : ce run traite le 13 juillet avec les valeurs par défaut. Sortie de SV_Summary, onglet **Sortie** sous la toile :

```
Fenêtre 2026-07-13 (jour ouvré) : 36 sessions facturées, 0 doublons écartés
```

5 Le déclencheur à fenêtre bascule

Tout publier. Puis PL_Charge_Window › **Ajouter un déclencheur** › **Nouveau/Modifier** › **+ Nouveau** :

Champ	Valeur
Nom	`TR_Window_Daily`
Type	**Fenêtre bascule**
Date de début (UTC)	**13/07/2026 00:00**
Périodicité	**24 Heures**
Date de fin (UTC)	coche **Spécifier une date de fin** : **16/07/2026 00:00**
Concurrence max	`1`
Nouvelle tentative	nombre `1`, intervalle `30`

OK, puis **windowStart** reçoit :

```
@trigger().outputs.windowStartTime
```

Gérer › **Déclencheurs** › **TR_Window_Daily** › **Dépendances** › **+ Nouveau** › **TR_Window_Daily** lui-même :

Décalage
```
-1.00:00:00
```

Taille
```
1.00:00:00
```

Tout publier. Le déclencheur découvre que ses trois fenêtres sont passées et les lance seul, l'une après l'autre.

La date de fin est indispensable. Sans elle, le déclencheur continuerait jour après jour jusqu'à aujourd'hui et s'arrêterait sur le 16 juillet, faute de fichier.

6 Vérifier, puis couper l'API

Surveillance › **Exécutions de pipeline** › **Déclenché** : trois runs **Réussite**, dans l'ordre. Le premier dure environ 3 minutes (démarrage de Spark), les suivants une trentaine de secondes. Sortie de **SV_Summary** de chacun :

```
Fenêtre 2026-07-13 (jour ouvré) : 36 sessions facturées, 0 doublons écartés
Fenêtre 2026-07-14 (jour férié) : 30 sessions facturées, 4 doublons écartés
Fenêtre 2026-07-15 (jour ouvré) : 33 sessions facturées, 3 doublons écartés
```

stchargeXXXX › **Navigateur de stockage** › **guide3** › **gold** › **2026-07-14** › **daily.csv** :

station_name	day_type	sessions	kwh	kwh_pleine	revenue_eur	holiday_discount_eur
Metz Gare	FERIE	10	344.64	0.0	115.29	32.38
Nancy Stanislas	FERIE	10	352.4	0.0	127.33	33.26
Thionville Centre	FERIE	10	353.63	0.0	112.1	23.41

Jour	Reçues	Facturées	kWh	Chiffre d'affaires	Remise férié	Doublons écartés
Lundi 13	36	36	943,36	405,46 €	0 €	aucun
Mardi 14	34	30	1 050,67	354,72 €	**89,05 €**	MET-071311, MET-071312, NAN-071312, THI-071312
Mercredi 15	36	33	893,51	395,81 €	0 €	MET-071410, NAN-071410, THI-071410

Le 14 juillet, en chiffres. C'est le jour où les bornes ont livré le plus d'énergie (1 050 kWh) et où le chiffre d'affaires est le plus bas : le tarif creux appliqué toute la journée a rendu 89,05 € aux clients, chiffrés station par station.

Le test qui prouve tout : couper l'API

PL_Charge_Window › **Ajouter un déclencheur** › **Déclencher maintenant**, en remplaçant les deux paramètres :

windowStart
```
2026-07-14T00:00:00Z
```

holidaysApiUrl
```
https://calendrier.api.gouv.fr/jours-feries/introuvable.json
```

WEB_Holidays passe au rouge après ses deux nouvelles tentatives, **LKP_HolidaysLocal** et **SV_HolidaysLocal** au vert, et le run finit en **Réussite** avec exactement le même bilan que le 14 juillet. Deux preuves en un run : la **résilience** (la panne d'une API externe ne casse pas la facturation) et l'**idempotence** (rejouer une fenêtre réécrit ses dossiers à l'identique, sans toucher aux autres jours).

Si quelque chose bloque

Symptôme	Remède
0 session facturée	Le **Flux de gauche** d'exNew doit être **dcToday**.
srcYesterday : chemin introuvable	Le jour zéro manque dans `silver/2026-07-12`.
SV_IsHoliday reste gris	Il manque une des quatre flèches bleues ou grises.
Fenêtre du 16 en échec	Date de fin absente : arrêtez le déclencheur.
Fenêtres 14 et 15 « En attente »	Normal pendant le 13. Si le 13 échoue : corrigez, puis **Réexécuter** sa fenêtre.
Aperçu des données sans valeur	Dans **Paramètres de débogage**, renseignez les paramètres du flux et de chaque jeu de données.

7 Tout fermer

Grâce à sa date de fin, le déclencheur s'est arrêté seul après le 15 juillet. Chaque fenêtre coûte surtout son data flow, quelques centimes.

Pour garder vos guides en sommeil

1. Vérifiez que le **Débogage du flux de données** est éteint.
2. **Gérer** › **Déclencheurs** : **TR_Window_Daily** à l'état **Arrêté**, puis **Tout publier**.

Pour effacer seulement ce guide : supprimez **TR_Window_Daily**, **PL_Charge_Window**, **DF_Charge_Window**, puis le conteneur `guide3`. Les guides n° 1 et n° 2 restent intacts.

Pour tout effacer : supprimez le groupe de ressources ci-dessous. Les trois guides disparaissent ensemble.

```
rg-adf-charge
```